Scrum Planning Package for RegimeLens

Starting from testing whether you can **apply Scrum as a framework** (not just describe it) by producing credible artefacts that demonstrate: sprint planning, sprint backlog creation, team communication, sprint execution controls, and the "last two activities" (Sprint Review + Sprint Retrospective) with outcomes. Scrum is designed as a lightweight framework that generates value through iterative, incremental delivery and empiricism (transparency, inspection, adaptation).

First, **Scrum correctness**: accountabilities, events, artefacts, and commitments reflect the Scrum Guide (e.g., Sprint Goal, Sprint Backlog composition, Daily Scrum purpose, and the intent/outcomes of Review and Retrospective).

Second, **realism**: credible agenda timings, capacity considerations, concrete acceptance criteria, and "how work will be done" broken down into implementable tasks. Scrum requires Developers to create the Sprint Backlog plan and adapt it daily toward the Sprint Goal.

Third, **tool operationalisation**: demonstrating how the artefacts would be tracked in **Azure DevOps Boards** via backlogs, iterations (sprints), and taskboards (including estimates and remaining work). In Azure Boards, a backlog is explicitly defined as a **prioritised list of work items** supporting scope management and collaboration, and teams have product, portfolio, and sprint backlogs.

Scrum Guide alignment that your submission must reflect

Your submission should visibly mirror the Scrum Guide's structure:

Scrum accountabilities are explicitly **Product Owner**, **Scrum Master**, and **Developers** within one Scrum Team.

Sprint Planning must address three topics: (1) **Why** the Sprint is valuable (Sprint Goal), (2) **What** can be done (selected Product Backlog items), and (3) **How** the chosen work will get done (decomposed planning by Developers).

The Sprint Backlog is not merely a list; it is composed of the **Sprint Goal (why)**, **selected Product Backlog items (what)**, and an **actionable plan (how)**. It is also updated throughout the Sprint as more is learned.

During the Sprint, Scrum sets clear guardrails: **no changes that endanger the Sprint Goal**, quality does not decrease, the Product Backlog is refined as needed, and scope may be clarified/renegotiated with the Product Owner as learning occurs.

The Daily Scrum purpose must align exactly: inspect progress toward Sprint Goal and adapt the Sprint Backlog, producing an actionable plan for the next day; it is a **15-minute event for Developers**.

The last two Scrum events and their outcomes must be described correctly: Sprint Review inspects the Sprint outcome and determines future adaptations with stakeholders; Sprint Retrospective plans ways to increase quality and effectiveness and may produce improvements added to the next Sprint Backlog.

Finally, the Definition of Done matters because work cannot be considered part of an Increment unless it meets the Definition of Done.

Translating RegimeLens MVP into a sprint-sized scope

A strong academic sprint plan starts with a clear **Product Goal** and a Sprint Goal that advances it. The Scrum Guide defines the Product Goal as a future state/target in the Product Backlog, used by the team to plan against.

For RegimeLens, your Product Goal can be expressed (academically) as:

Product Goal (RegimeLens): Provide an explainable, single-asset market regime detection workflow that allows users to select an asset, compute indicators, detect regimes, and visualise regimes with interpretability.

From your described MVP flow, the Sprint should deliver an end-to-end "thin slice" across: asset input → OHLCV retrieval → indicators → clustering → chart overlay → basic explanation note. This directly supports Scrum's emphasis on producing a usable Increment each Sprint, inspected at Sprint Review.

Because Scrum allows techniques within the framework (it is intentionally incomplete), using user stories, story points, and Planning Poker is acceptable as long as you keep the official Scrum elements correct.

Sprint Planning artefacts for RegimeLens

This section is written to be copied into your PDF form as-is.

Sprint Planning agenda

Sprint Planning initiates the Sprint and must be the collaborative work of the entire Scrum Team; the Sprint Goal must be finalised before Sprint Planning ends.

Sprint length: 2 weeks (common cadence)
Sprint Goal: Deliver an MVP vertical slice enabling a user to choose an asset, retrieve

OHLCV, compute baseline indicators, cluster regimes, and visualise regimes on a chart, with a minimal explanatory summary.

Timebox	Agenda item	Purpose	Scrum Guide mapping
0–10 min	Opening + working agreements reminder	Reinforce Sprint Goal focus, timeboxes, Definition of Done	Sprint guardrails + DoD
10–25 min	Review top Product Backlog items	Ensure backlog items are understood and ordered	Product Backlog management
25–45 min	Topic 1: Why is this Sprint valuable?	Draft and finalise Sprint Goal	Sprint Planning Topic 1
45–70 min	Topic 2: What can be Done this Sprint?	Developers forecast/select items based on capacity and DoD	Sprint Planning Topic 2
70–95 min	Topic 3: How will the chosen work get done?	Break selected items into tasks (≤1 day), define approach	Sprint Planning Topic 3
95–110 min	Estimation + capacity check	Confirm forecast is feasible; identify dependencies/risks	Capacity consideration
110–120 min	Confirm Sprint Backlog + commitment	Confirm Sprint Goal + selected items + plan	Sprint Backlog definition

Sprint Planning meeting simulation

This is a realistic simulation aligned to Scrum accountabilities.

Participants:
Product Owner (PO), **Scrum Master (SM)**, **Developers (Dev 1, Dev 2)**.

SM: "Welcome. Our outcome today is a Sprint Backlog: Sprint Goal, selected backlog items, and a plan. We'll keep focus on what makes this Sprint valuable and confirm Definition of Done."

PO: "Value this Sprint is an end-to-end RegimeLens MVP slice: asset selection → data retrieval → indicators → regimes → chart overlay. Stakeholders need interpretability, so the visual output must be clear."

Dev 1: "For OHLCV retrieval, do we use a live API first or a static dataset for reliability?"

PO: "Use a public source for realism, but we need a fallback dataset if API fails so we don't block the Sprint Goal." (This protects the Sprint Goal from risks/endangering changes.)

Dev 2: “For regime detection, we’ll start with K-Means clustering as the MVP clustering approach. We’ll design the pipeline so we can later swap in other methods.”

SM: “Great. Let’s finalise the Sprint Goal: a user can pick an asset and see colour-coded regimes on a chart. Then Developers will forecast how many PBIs we can complete, considering capacity and Definition of Done.”

Developers: “We forecast the following PBIs as achievable within two weeks, assuming tasks are split to about one day each, and that we keep the scope stable.”

Sprint Backlog

Scrum requires the Sprint Backlog to include the Sprint Goal, selected items (“what”), and an actionable plan (“how”). It is owned and updated by Developers through the Sprint.

Estimation method: Story points using Planning Poker (consensus-based; highlights uncertainty and surfaces risk early).

PBI ID	Product Backlog Item (user story form)	Priority	Estimate (SP)	Acceptance criteria (Done means…)
RL-01	As a user, I can enter/select a stock/crypto ticker so I can run regime detection on it.	High	3	Valid ticker is accepted; invalid ticker shows clear error; selected ticker is stored for downstream steps.
RL-02	As a user, I can retrieve historical OHLCV for the selected asset so indicators can be computed.	High	5	OHLCV dataset loads for chosen date range; missing/empty response handled with fallback; data schema validated.
RL-03	As a user, I can generate baseline indicators (volatility + moving averages) so regimes can be distinguished.	High	5	Indicators computed correctly for dataset; values align with expected ranges; indicator features available for clustering.
RL-04	As a user, I can detect at least 3 market regimes via clustering so I can see different states.	High	8	Clustering produces ≥3 regimes; each data point receives a regime label; pipeline persists labels for charting.
RL-05	As a user, I can visualise regimes on a price chart so I	High	8	Chart overlays regimes using distinct colours; legend explains

PBI ID	Product Backlog Item (user story form)	Priority	Estimate (SP)	Acceptance criteria (Done means…)
	can interpret market states easily.			regimes; transitions are visually identifiable.
RL-06 (stretch)	As a user, I can view a short explanation per regime so I understand what the regime represents.	Medium	3	For each regime: short label + indicator summary (e.g., "high vol / downward trend"); explanation displayed with chart.

Planned capacity: 29–32 SP for a 2-week Sprint (team-specific; justify with availability).
Initial forecast: RL-01 to RL-05 (29 SP). RL-06 is stretch if capacity remains after stabilisation/testing.

Task breakdown (actionable plan): tasks should be ≤1 day where possible.

PBI	Task	Owner	Tooling notes
RL-01	Build ticker input + validation	Dev 1	UI component + validation rules
RL-02	Implement OHLCV retrieval + fallback dataset	Dev 1	Add retry + deterministic sample data
RL-02	Data cleaning + schema validation	Dev 2	Validate columns, missing values
RL-03	Compute moving averages	Dev 2	Parameterised windows (e.g., 20/50)
RL-03	Compute rolling volatility	Dev 1	Standard deviation of returns
RL-04	Feature matrix build + scaling	Dev 2	Standardise features
RL-04	K-Means training + regime labelling	Dev 2	Deterministic seed for reproducibility
RL-05	Chart overlay (colour by regime)	Dev 1	Legend + annotations

PBI	Task	Owner	Tooling notes
RL-05	Integration testing of full pipeline	Dev 1 + Dev 2	Ensure end-to-end run works
RL-06	Regime explanation template	Dev 2	Summary from indicator stats

Sprint communication document and Azure DevOps Boards implementation

This section both (a) provides the "specific shared document" required by your rubric, and (b) shows how you would implement it in Azure DevOps Boards.

Sprint communication document

Scrum depends on transparency and effective inspection/adaptation; your communication plan should make transparency operational (who communicates what, when, and where).

Document title: RegimeLens Sprint 1 Communication Plan
Sprint dates: [enter your course dates]
Sprint Goal: [copy Sprint Goal from above]

Communication channels and response expectations

Context	Channel	Expected response time	Owner
Work tracking (source of truth)	Azure DevOps Boards	Same day updates	Developers
Daily coordination	Stand-up (Teams/Zoom/in-person)	Live	Scrum Master
Blockers/impediments	Teams message + flag work item	≤2 hours during working time	Scrum Master
Requirements clarification	PO check-in (async)	≤24 hours	Product Owner
Code collaboration	Pull requests + review comments	≤24 hours	Developers

Ceremony schedule

Event	Cadence	Duration	Required attendees	Output
Sprint Planning	Day 1	2 hours	Whole Scrum Team	Sprint Goal + Sprint Backlog
Daily Scrum	Daily	15 min	Developers (PO/SM if acting as Devs)	Updated plan for next day
Backlog refinement (activity)	Mid-sprint	45 min	PO + Developers (+SM facilitate)	Better-prepared PBIs
Sprint Review	End of sprint	60 min	Scrum Team + stakeholders	Adapted Product Backlog
Sprint Retrospective	End of sprint	60 min	Whole Scrum Team	Improvement plan (may enter next Sprint Backlog)

Definition of Done (Sprint 1)

To ensure quality does not decrease during the Sprint, the team will treat "Done" as:

1. Code committed and peer-reviewed
2. Minimum automated checks pass (unit tests for indicator functions; smoke test for pipeline)
3. Feature works end-to-end on a representative dataset
4. User-facing output is understandable (chart + legend + short labels)
5. Work item updated in Azure DevOps to Done with evidence (screenshots/log output attached)

This aligns with Scrum's requirement that work is only part of the Increment when it meets the Definition of Done.

How to represent this in Azure DevOps Boards

This part maps your sprint artefacts into Azure Boards concepts and fields.

Choose a process template:

If you select the **Scrum** process in Azure DevOps, your backlog uses **Product Backlog Items** and you can decompose them into tasks on the taskboard; Scrum-mode tasks track Remaining Work only.

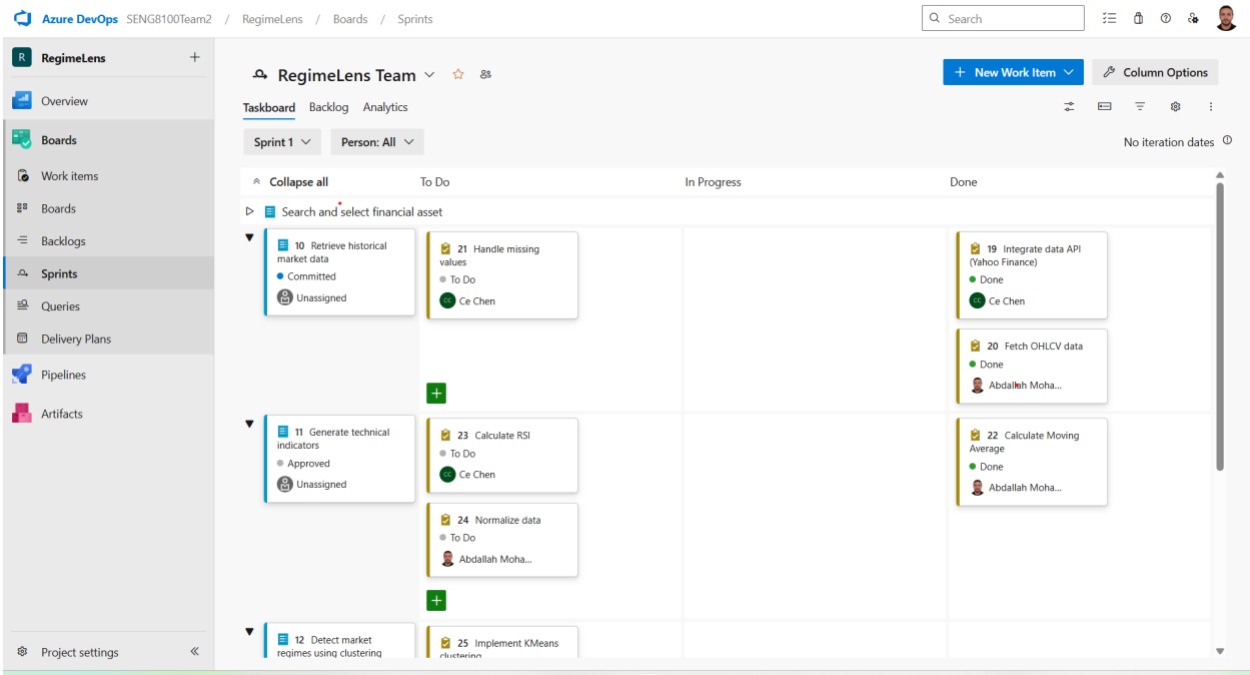
Backlog structure to use (typical):

Epic → Feature → Product Backlog Item → Task (and optionally Bug).

Set sprint cadence and dates:
Azure Boards implements sprints as Iteration Paths with start/end dates; many teams choose a 2–3 week cadence.

Field mapping for estimation and tracking:
In Azure DevOps, estimation fields differ by process. In Scrum projects, **Effort** is used on PBIs/Bugs as a subjective sizing measure, while tasks focus on Remaining Work.

Board hygiene and ordering:
Azure Boards describes the backlog as a prioritised list that helps manage scope and collaboration, and reordering updates underlying ordering fields (e.g., Backlog Priority for Scrum).



Sprint execution controls and sprint backlog management

Sprint execution quality is largely determined by whether you run the Sprint in a way that preserves empiricism and protects the Sprint Goal.

Sprint guardrails you must mention (Scrum Guide):
During the Sprint, there should be no changes that endanger the Sprint Goal, quality should not decrease, backlog refinement occurs as needed, and scope can be clarified/renegotiated with the Product Owner as learning occurs. These are foundational sprint execution statements and should be echoed directly in your assignment response.

Backlog management in the Sprint (practical and Scrum-correct):
The Sprint Backlog is a real-time plan by Developers and is updated throughout the Sprint

“as more is learned,” which is why your Daily Scrum should result in an actionable plan for the next day.

Operational best practices that fit the Scrum framework (and help grading):
Planning and estimation techniques are optional within Scrum but are commonly used to create transparency and surface risk early. Planning Poker is a consensus technique designed to reveal uncertainty and implementation risk through discussion.

Continuous integration supports fast feedback and reduces painful integration risk; Martin Fowler’s classic description emphasises that frequent integration is enabled by automation (build + significant testing) that yields a clear “build worked: yes/no” signal.

Branch policies and pull-request reviews are a pragmatic quality control that align with the Scrum requirement that quality does not decrease; Azure DevOps branch policies can require PRs and a minimum number of reviewers, preventing direct pushes to mainline.

Sprint Review and Sprint Retrospective outcomes for RegimeLens

This section addresses the assignment requirement to discuss the last two activities and their outcomes, and provides realistic outcomes for RegimeLens.

Sprint Review

Scrum definition: The purpose of the Sprint Review is to inspect the Sprint outcome and determine future adaptations; the Scrum Team presents results to key stakeholders and discusses progress toward the Product Goal, and the Product Backlog may be adjusted.

RegimeLens Sprint Review agenda (60 minutes):
Demonstrate (1) asset selection, (2) OHLCV retrieval, (3) indicators, (4) regimes shown on a chart. Then capture stakeholder feedback and update the Product Backlog ordering.

Example Sprint Review outcomes (credible):
Stakeholders confirm the value of regime overlay on price charts (validates the interpretability hypothesis). Product Owner adds/refines backlog items: “interactive tooltips per regime” and “short plain-English regime summary,” and reorders them higher because stakeholders need clarity at the point of interpretation. This is consistent with Scrum’s expectation that the Product Backlog may be adapted after inspection.

Sprint Retrospective

Scrum definition: The purpose is to plan ways to increase quality and effectiveness by inspecting how the last Sprint went (people, interactions, process, tools, and Definition of Done) and selecting the most helpful improvements; improvements may be added to the next Sprint Backlog.

RegimeLens Retrospective outcomes (example):
The team identifies that data-source instability caused delays; improvement: add a stable sample dataset and implement schema validation earlier. The team also notes that chart styling decisions were debated late; improvement: agree a simple chart standard in the Definition of Done. These improvements are actionable and can enter the next Sprint Backlog as tasks (matching Scrum Guide guidance).